

✓ Phase 1: Big Data Environment Setup & Data Acquisition

Install + Imports

```
# Install
!pip install datasets pandas

# Imports
from datasets import load_dataset
import pandas as pd
import os
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.29.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.0.2)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocessing<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3)
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (1.11.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (26.1)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)
Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->da)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datas)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (0.24.2)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datas)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.13)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fss)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fss)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fs)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fs)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fs)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fs)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24.0->
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggi)
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.0->dat)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.0->dat)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.0->dat)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.0->dat)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggi)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggi)
Requirement already satisfied: mdurl~0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->
```

Load Dataset

```
dataset = load_dataset("KFUPM-JRCAI/arabic-generated-abstracts")

print("Dataset loaded successfully!")
print(dataset)
```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
README.md:      8.06k/? [00:00<00:00, 549kB/s]

data/by_polishing-00000-of-00001.parquet: 100%                8.49M/8.49M [00:01<00:00, 42.4MB/s]

data/from_title-00000-of-00001.parquet: 100%                 6.90M/6.90M [00:00<00:00, 33.5MB/s]

data/from_title_and_content-00000-of-000(...): 100%          7.01M/7.01M [00:00<00:00, 34.9MB/s]

Generating by_polishing split: 100%                          2851/2851 [00:00<00:00, 2943.77 examples/s]

Generating from_title split: 100%                             2963/2963 [00:00<00:00, 10437.70 examples/s]

Generating from_title_and_content split: 100%                2574/2574 [00:00<00:00, 3416.95 examples/s]

Dataset loaded successfully!
DatasetDict({
  by_polishing: Dataset({
    features: ['original_abstract', 'allam_generated_abstract', 'jais_generated_abstract', 'llama_generated_abstract', 'open
    num_rows: 2851
  })
  from_title: Dataset({
    features: ['original_abstract', 'allam_generated_abstract', 'jais_generated_abstract', 'llama_generated_abstract', 'open
    num_rows: 2963
  })
  from_title_and_content: Dataset({
    features: ['original_abstract', 'allam_generated_abstract', 'jais_generated_abstract', 'llama_generated_abstract', 'open
    num_rows: 2574
  })
})

```

Inspect Structure

```

# Check one sample from each split
print(dataset['by_polishing'][0])
print(dataset['from_title'][0])
print(dataset['from_title_and_content'][0])

```

```

{'original_abstract': '؟ الأندلسيين الفكر التربوي والإنتاج الفكري الأندلسيين ؟'}
{'original_abstract': '؟ الأندلسيين الفكر التربوي والإنتاج الفكري الأندلسيين ؟'}
{'original_abstract': '؟ الأندلسيين الفكر التربوي والإنتاج الفكري الأندلسيين ؟'}

```

Convert to ML Dataset

```

data = []

splits = ['by_polishing', 'from_title', 'from_title_and_content']

for split in splits:
    for item in dataset[split]:

        # Human text
        data.append({
            "text": item["original_abstract"],
            "label": 0
        })

        # AI texts
        data.append({"text": item["allam_generated_abstract"], "label": 1})
        data.append({"text": item["jais_generated_abstract"], "label": 1})
        data.append({"text": item["llama_generated_abstract"], "label": 1})
        data.append({"text": item["openai_generated_abstract"], "label": 1})

df = pd.DataFrame(data)

print("DataFrame created!")

DataFrame created!

```

Exploration

```
print("\nSample Data:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nLabel Distribution:")
print(df['label'].value_counts())
```

Sample Data:

		text	label
0	0	كثيرا ما ارتبطت المصادر التاريخية في الأندلس خ...	
1	1	يتناول هذا البحث موضوع التعليم بين النساء الآن...	
2	1	تدرس هذه الدراسة دور المرأة في التعليم في الآن...	
3	1	يقدم هذا البحث دراسة شاملة حول حالة التعليم ع...	
4	1	صور نظام التعليم عند المرأة الأندلسية تستند إل...	

Dataset Shape:
(41940, 2)

Label Distribution:

label	
1	33552
0	8388

Name: count, dtype: int64

Save Dataset

```
# Create folder
os.makedirs("data", exist_ok=True)

# Save file
df.to_csv("data/data.csv", index=False)

print("Dataset saved successfully at: data/data.csv")
```

Dataset saved successfully at: data/data.csv

```
# Install PySpark (Colab)
!pip install pyspark
```

Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.2)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)

Spark Session

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("AI Text Detection") \
    .getOrCreate()

print("Spark Session Created")
```

Spark Session Created

```
spark_df = spark.read.csv("data/data.csv", header=True, inferSchema=True)

spark_df.show(5)
```

```
+-----+-----+
|          text|label|
+-----+-----+
|0|    كثيرا ما ارتبطت ا...|
|1|    يتناول هذا البحث...|
|1|    تدرس هذه الدراس...|
|"1"|    يقدم هذا البحث...|
|"1"|    صور نظام التعليم...|
+-----+-----+
only showing top 5 rows
```

Schema: count

```
Schema + Count
```

```
print("Schema:")
spark_df.printSchema()
```

```
Schema:
root
 |-- text: string (nullable = true)
 |-- label: string (nullable = true)
```

```
print("Count:")
print(spark_df.count())
```

```
Count:
51743
```

```
print("Label Distribution (Spark):")
spark_df.groupBy("label").count().show()
```

```
Label Distribution (Spark):
+-----+-----+
|          label|count|
+-----+-----+
| 3 | وفقا للمنهج الأيس...|
| 3 | تحديد الرسالة ا...|
| 0 | 8357|
| NULL | 9791|
| 2 | هذا ما جعلنا نحا...|
| 01 1 | بين التكم وال...|
| 01 3 | ماعدا فرضية وا...|
| 3 | والنمو المعرفي و...|
| 3 | أو جيل |
| 2 | يطلق على الذات و...|
| 01 3 | بين التكم وال...|
| 3 | وعموما يوضح الب...|
| 1 | إلى استكشاف الصر...|
| 1 | 33492|
| 3 | عماد الدين خليل |
| 3 | كما يسلط الضوء ع...|
| M54 | 1|
| P2." | 1|
| P34 | 1|
| J53 | 1|
+-----+-----+
only showing top 20 rows
```

```
df.to_csv("data/data.csv", index=False, encoding="utf-8-sig")
```

```
spark_df = spark.read.option("header", True) \
    .option("multiline", True) \
    .option("quote", "'') \
    .option("escape", "'') \
    .csv("data/data.csv")
```

```
from pyspark.sql.functions import col

spark_df = spark_df.withColumn("label", col("label").cast("int"))
```

Correct Spark label distribution

```
spark_df.groupBy("label").count().show()
```

```
+-----+-----+
|label|count|
+-----+-----+
| 1 | 33552|
| 0 | 8388|
+-----+-----+
```

```
spark_df.write.mode("overwrite").parquet("data/processed_data.parquet")
```

Phase 2: Distributed Data Preprocessing & Storage

```
from pyspark.sql import SparkSession

# Initialize Spark
spark = SparkSession.builder.appName("Phase2").getOrCreate()

# Load CSV into Spark
spark_df = spark.read.option("header", True) \
    .option("multiline", True) \
    .option("quote", "'') \
    .option("escape", "'') \
    .csv("data/data.csv")

spark_df.show(5)
```

```
+-----+-----+
|          text|label|
+-----+-----+
|0|    كثيرا ما ارتبطت ا...|
|1|    يتناول هذا البحث...|
|1|    تدرس هذه الدراسة...|
|1|    يقدم هذا البحث د...|
|1|    صور نظام التعليم...|
+-----+-----+
only showing top 5 rows
```

```
import re

# Normalize Arabic characters
def normalize_arabic(text):
    text = re.sub("[\i\i\i]", "\i", text)
    text = re.sub("ى", "ي", text)
    text = re.sub("ذ", "ذ", text)
    text = re.sub("ئ", "ي", text)
    text = re.sub("ة", "ه", text)
    return text

# Remove diacritics
def remove_diacritics(text):
    arabic_diacritics = re.compile("""
        `|'|''|'|_|
    """, re.VERBOSE)
    return re.sub(arabic_diacritics, '', text)

# Basic cleaning
def clean_text(text):
    text = remove_diacritics(text)
    text = normalize_arabic(text)
    text = re.sub(r'^\w\s', '', text)
    text = re.sub(r'\d+', '', text)
    return text
```

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

# Apply cleaning using UDF
clean_udf = udf(lambda x: clean_text(x) if x else "", StringType())

spark_df = spark_df.withColumn("clean_text", clean_udf(spark_df["text"]))

spark_df.show(5)
```

```
+-----+-----+-----+
|          text|label|      clean_text|
+-----+-----+-----+
|0|    كثيرا ما ارتبطت ا...|    كثيرا ما ارتبطت ا...|
|1|    يتناول هذا البحث...|    يتناول هذا البحث...|
|1|    تدرس هذه الدراسة...|    تدرس هذه الدراسة...|
|1|    يقدم هذا البحث د...|    يقدم هذا البحث د...|
|1|    صور نظام التعليم...|    صور نظام التعليم...|
+-----+-----+-----+
only showing top 5 rows
```

```
# Simple Arabic stopwords list
arabic_stopwords = set([
    "قد", "كان", "مع", "كما", "ذلك", "هذه", "هذا", "عن", "الى", "و", "على", "من", "في"
])

def remove_stopwords(text):
    words = text.split()
    filtered = [w for w in words if w not in arabic_stopwords]
    return " ".join(filtered)

stop_udf = udf(lambda x: remove_stopwords(x), StringType())

spark_df = spark_df.withColumn("clean_text", stop_udf(spark_df["clean_text"]))

spark_df.show(5)
```

```
+-----+-----+-----+
|          text|label|          clean_text|
+-----+-----+-----+
|0|كثيرا ما ارتبطت ا...|...|كثيرا ما ارتبطت ا...|
|1|يتناول هذا البحث...|...|يتناول هذا البحث...|
|1|تدرس هذه الدراسة...|...|تدرس هذه الدراسة...|
|1|يقدم هذا البحث دراسة...|...|يقدم هذا البحث دراسة...|
|1|صور نظام التعليم...|...|صور نظام التعليم...|
+-----+-----+-----+
only showing top 5 rows
```

```
# Simple stemming (first 4 chars)
def simple_stem(text):
    words = text.split()
    stemmed = [w[:4] for w in words]
    return " ".join(stemmed)

stem_udf = udf(lambda x: simple_stem(x), StringType())

spark_df = spark_df.withColumn("processed_text", stem_udf(spark_df["clean_text"]))

spark_df.select("text", "processed_text").show(5)
```

```
+-----+-----+-----+
|          text|          processed_text|
+-----+-----+-----+
|...|كثيرا ما ارتبطت ا...|...|كثيرا ما ارتبطت ا...|
|...|يتناول هذا البحث...|...|يتناول هذا البحث...|
|...|تدرس هذه الدراسة...|...|تدرس هذه الدراسة...|
|...|يقدم هذا البحث دراسة...|...|يقدم هذا البحث دراسة...|
|...|صور نظام التعليم...|...|صور نظام التعليم...|
+-----+-----+-----+
only showing top 5 rows
```

```
# Save in Parquet format
spark_df.write.mode("overwrite").parquet("data/processed_phase2.parquet")
```

```
from pyspark.sql.functions import explode, split

# Tokenize words
words_df = spark_df.select(explode(split("processed_text", " ")).alias("word"))
words_df = words_df.filter(words_df.word != "")

# Count frequency
word_counts = words_df.groupBy("word").count().orderBy("count", ascending=False)

word_counts.show(10)
```

```
+----+-----+
|word|count|
+----+-----+
|علي|99134|
|الي|75976|
|البيع|49607|
|الدر|48972|
|الاس|39865|
|ان|38408|
|التي|33133|
|الجز|30468|
|المث|29978|
|بين|29650|
```

```
+-----+
only showing top 10 rows
```

```
# Calculate TTR
total_words = words_df.count()
unique_words = words_df.select("word").distinct().count()

ttr = unique_words / total_words

print("Total Words:", total_words)
print("Unique Words:", unique_words)
print("TTR:", ttr)
```

```
Total Words: 3937649
Unique Words: 23129
TTR: 0.005873809473622459
```

N-Gram analysis

```
from pyspark.ml.feature import NGram

words_split = spark_df.withColumn("words", split("processed_text", " "))

ngram = NGram(n=2, inputCol="words", outputCol="bigrams")

ngram_df = ngram.transform(words_split)

ngram_df.select("bigrams").show(5, truncate=False)
```

```
+-----+
|bigrams|
+-----+
|عند التّع عند، عند المر، المر الان، الان وكيف وكيف كانت كانت تأخذ تأخذ ففون ففون العل العل وما مدي مدي اسها الفك الفك التز التز والا الفك الفك الان|
|سعي، يسعي النج، النج الي، الي فهم فهم كيفي كيفي تلقى تلقى المر المر الان، الان للعل للعل ومدي ومدي تأثي تأثي علي علي الفك الفك التز التز والا الفك الفك الان|
|الي، الي فهم فهم حاله حاله التّع التّع بين بين النّس النّس الان، الان وكيف كن كن يكتس يكتس المع المع ودور الدور الفك الفك التز التز والا الفك الفك المن|
|و للمر الان، الان يعتب يعتب النج، النج مساه مساه قيمه قيمه فهم فهم تاري تاري التّع التّع عند المر المر الان، الان وتأث وتأث الفك الفك التز التز والف والف الحص|
|متلق، متلق للعل للعل بل بل كن كن ايضا منتج منتج ومسا، ومسا المع المع مما مما يفسح يفسح المع المع لفهم لفهم اكبر اكبر لدور لدور اثرا اثرا التّق التّق الان|
+-----+
```

only showing top 5 rows

```
# Convert Spark DataFrame column to RDD
text_rdd = spark_df.select("processed_text").rdd.flatMap(lambda x: x)

# MapReduce word count
word_counts_mr = text_rdd.flatMap(lambda line: line.split()) \
    .map(lambda word: (word, 1)) \
    .reduceByKey(lambda a, b: a + b)

print(word_counts_mr.take(10))
```

```
[('ب', 466), ('منها', 2484), ('خاصه', 3289), ('الان', 19104), ('التا', 13042), ('المص', 6559), ('ارتب', 1190), ('ما', 8058), ('كثير', 733)]
```

Phase 3: Scalable Feature Engineering & Distributed Modeling

```
from pyspark.sql.functions import col
from pyspark.sql.functions import udf
from pyspark.sql.types import IntegerType
import re
```

Feature 6 (Multiple Elongations)

```
# Count multiple elongations (e.g., loooong letters)

def count_multiple_elongations(text):
    return len(re.findall(r'(\d{2,})', text))

elong_udf = udf(lambda x: count_multiple_elongations(x), IntegerType())

spark_df = spark_df.withColumn("feat_6_elong", elong_udf(col("text")))
```

Feature 29 (Semicolons)

```
# Count semicolons

def count_semicolons(text):
    return text.count(";")

semi_udf = udf(lambda x: count_semicolons(x), IntegerType())

spark_df = spark_df.withColumn("feat_29_semicolon", semi_udf(col("text")))
```

Feature 52 (Interjections)

```
# Arabic interjections list

interjections = ["يا", "ا", "و", "ا", "و"]

def count_interjections(text):
    words = text.split()
    return sum(1 for w in words if w in interjections)

interj_udf = udf(lambda x: count_interjections(x), IntegerType())

spark_df = spark_df.withColumn("feat_52_interj", interj_udf(col("text")))
```

Feature 75 (Active Voice Approx)

```
# Simple approximation: count verbs-like patterns

def count_active_voice(text):
    words = text.split()
    return sum(1 for w in words if w.startswith("ي") or w.startswith("ت"))

active_udf = udf(lambda x: count_active_voice(x), IntegerType())

spark_df = spark_df.withColumn("feat_75_active", active_udf(col("text")))
```

Feature 98 (BERT Simplified)

```
# Approximate BERT feature (text length proxy)

def bert_proxy(text):
    return len(text.split())

bert_udf = udf(lambda x: bert_proxy(x), IntegerType())

spark_df = spark_df.withColumn("feat_98_bert", bert_udf(col("text")))
```

Check Features

```
spark_df.select(
    "feat_6_elong",
    "feat_29_semicolon",
    "feat_52_interj",
    "feat_75_active",
    "feat_98_bert"
).show(5)
```

feat_6_elong	feat_29_semicolon	feat_52_interj	feat_75_active	feat_98_bert
0	0	0	5	119
0	0	0	6	68
0	0	0	6	66
0	0	0	9	100
0	0	0	14	173

only showing top 5 rows

Tokenize + TF-IDF (Spark ML)

```
from pyspark.ml.feature import Tokenizer, StopWordsRemover, HashingTF, IDF

# Tokenize
tokenizer = Tokenizer(inputCol="processed_text", outputCol="words")

# (optional) extra stopwords remove at ML stage
remover = StopWordsRemover(inputCol="words", outputCol="filtered_words")

# TF
hashingTF = HashingTF(inputCol="filtered_words", outputCol="tf_features", numFeatures=10000)

# IDF
idf = IDF(inputCol="tf_features", outputCol="tfidf_features")

# Apply pipeline steps
df_tokens = tokenizer.transform(spark_df)
df_filtered = remover.transform(df_tokens)
df_tf = hashingTF.transform(df_filtered)
idf_model = idf.fit(df_tf)
df_tfidf = idf_model.transform(df_tf)

df_tfidf.select("tfidf_features").show(5)
```

```
+-----+
|      tfidf_features|
+-----+
|(10000,[36,64,218...|
|(10000,[36,218,34...|
|(10000,[83,249,50...|
|(10000,[218,500,8...|
|(10000,[218,249,2...|
+-----+
only showing top 5 rows
```

Combine ALL features

```
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
    inputCols=[
        "tfidf_features",
        "feat_6_elong",
        "feat_29_semicolon",
        "feat_52_interj",
        "feat_75_active",
        "feat_98_bert"
    ],
    outputCol="features"
)

final_df = assembler.transform(df_tfidf)

final_df.select("features", "label").show(5)
```

```
+-----+-----+
|      features|label|
+-----+-----+
|(10005,[36,64,218...|    0|
|(10005,[36,218,34...|    1|
|(10005,[83,249,50...|    1|
|(10005,[218,500,8...|    1|
|(10005,[218,249,2...|    1|
+-----+-----+
only showing top 5 rows
```

Label fix

```
from pyspark.sql.functions import col

final_df = final_df.withColumn("label", col("label").cast("int"))
```

Train / Test Split

```
train_df, temp_df = final_df.randomSplit([0.7, 0.3], seed=42)
val_df, test_df = temp_df.randomSplit([0.5, 0.5], seed=42)

print("Train:", train_df.count())
print("Validation:", val_df.count())
print("Test:", test_df.count())
```

```
Train: 29438
Validation: 6370
Test: 6132
```

Logistic Regression (Baseline)

```
from pyspark.ml.classification import LogisticRegression

lr = LogisticRegression(featuresCol="features", labelCol="label")

lr_model = lr.fit(train_df)

lr_predictions = lr_model.transform(test_df)
```

Random Forest

```
from pyspark.ml.classification import RandomForestClassifier

rf = RandomForestClassifier(featuresCol="features", labelCol="label")

rf_model = rf.fit(train_df)

rf_predictions = rf_model.transform(test_df)
```

SVM

```
from pyspark.ml.classification import LinearSVC

svm = LinearSVC(featuresCol="features", labelCol="label")

svm_model = svm.fit(train_df)

svm_predictions = svm_model.transform(test_df)
```

Accuracy + F1 evaluation

```
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction")

# Accuracy
print("LR Accuracy:", evaluator.evaluate(lr_predictions, {evaluator.metricName: "accuracy"}))
print("RF Accuracy:", evaluator.evaluate(rf_predictions, {evaluator.metricName: "accuracy"}))
print("SVM Accuracy:", evaluator.evaluate(svm_predictions, {evaluator.metricName: "accuracy"}))

# F1 Score
print("LR F1:", evaluator.evaluate(lr_predictions, {evaluator.metricName: "f1"}))
print("RF F1:", evaluator.evaluate(rf_predictions, {evaluator.metricName: "f1"}))
print("SVM F1:", evaluator.evaluate(svm_predictions, {evaluator.metricName: "f1"}))

LR Accuracy: 0.9657534246575342
RF Accuracy: 0.8007175472928898
SVM Accuracy: 0.973091976516634
LR F1: 0.966350782755693
RF F1: 0.7124325415524592
SVM F1: 0.9735649084002084
```

✓ Phase 4: Stream Processing & Comprehensive Evaluation

Create streaming folder

```
import os

# Create folder for streaming simulation
os.makedirs("/content/stream_data", exist_ok=True)

print("Streaming folder ready!")
```

Streaming folder ready!

Start Streaming Source

```
# File-based streaming source
stream_df = spark.readStream \
    .format("text") \
    .option("maxFilesPerTrigger", 1) \
    .load("/content/stream_data")

print("Streaming Started")
print("Is Streaming:", stream_df.isStreaming)
```

Streaming Started
Is Streaming: True

Rename Column

```
from pyspark.sql.functions import col

# Rename default column
stream_df = stream_df.withColumnRenamed("value", "text")
```

Apply Preprocessing

```
# Apply same preprocessing pipeline
stream_df = stream_df.withColumn("clean_text", clean_udf(col("text")))

stream_df = stream_df.withColumn(
    "processed_text",
    stem_udf(col("clean_text"))
)
```

TF-IDF Transformation

```
# Tokenization + TF-IDF
stream_tokens = tokenizer.transform(stream_df)

stream_filtered = remover.transform(stream_tokens)

stream_tf = hashingTF.transform(stream_filtered)

stream_tfidf = idf_model.transform(stream_tf)
```

Add Custom Features

```
# Add stylometric features
stream_features = stream_tfidf \
    .withColumn("feat_6_elong", elong_udf(col("text"))) \
    .withColumn("feat_29_semicolon", semi_udf(col("text"))) \
    .withColumn("feat_52_interj", interj_udf(col("text"))) \
    .withColumn("feat_75_active", active_udf(col("text"))) \
    .withColumn("feat_98_bert", bert_udf(col("text")))
```

Assemble Features

```
# Create final feature vector
stream_final = assembler.transform(stream_features)
```

Real-Time Prediction (Best Model)

```
# Use best-performing model (SVM)
predictions = svm_model.transform(stream_final)
```

Start Streaming Output

```
# Start prediction stream
query = predictions.select("text", "prediction") \
    .writeStream \
    .format("console") \
    .option("truncate", False) \
    .start()

print("Streaming Query Running...")
```

Streaming Query Running...

Simulate Live Incoming Data

```
# Add new streaming file
with open("/content/stream_data/live_text_1.txt", "w", encoding="utf-8") as f:
    f.write("هذا نص عربي جديد للاختبار\n")

print("New streaming file added!")
```

New streaming file added!

Stop Stream (after few seconds)

```
import time

# Wait to process stream
time.sleep(10)

# Stop stream
query.stop()

print("Streaming Stopped")
```

Streaming Stopped

ROC-AUC Evaluation

```
from pyspark.ml.evaluation import BinaryClassificationEvaluator

# ROC-AUC evaluator
evaluator_auc = BinaryClassificationEvaluator(labelCol="label")

print("LR AUC:", evaluator_auc.evaluate(lr_predictions))
print("RF AUC:", evaluator_auc.evaluate(rf_predictions))
print("SVM AUC:", evaluator_auc.evaluate(svm_predictions))
```

LR AUC: 0.9829234549701438
RF AUC: 0.9464322454735763
SVM AUC: 0.9911358373150938

Confusion Matrix

```
print("Logistic Regression Confusion Matrix")

lr_predictions.groupBy("label", "prediction").count().show()
```

```
Logistic Regression Confusion Matrix
+-----+-----+-----+
|label|prediction|count|
+-----+-----+-----+
| 1|      0.0|   166|
|  0|      0.0|  1180|
```

```
| 1| 1.0| 4742|
| 0| 1.0| 44|
+---+-----+-----+
```

```
print("Random Forest Confusion Matrix")

rf_predictions.groupBy("label", "prediction").count().show()
```

```
Random Forest Confusion Matrix
+---+-----+-----+
|label|prediction|count|
+---+-----+-----+
| 0| 0.0| 2|
| 1| 1.0| 4908|
| 0| 1.0| 1222|
+---+-----+-----+
```

```
print("SVM Confusion Matrix")

svm_predictions.groupBy("label", "prediction").count().show()
```

```
SVM Confusion Matrix
+---+-----+-----+
|label|prediction|count|
+---+-----+-----+
| 1| 0.0| 144|
| 0| 0.0| 1203|
| 1| 1.0| 4764|
| 0| 1.0| 21|
+---+-----+-----+
```

Batch Processing Time

```
import time

# Measure batch processing time
start = time.time()

svm_model.transform(test_df).count()

end = time.time()

print("Batch Processing Time:", end - start, "seconds")
```

```
Batch Processing Time: 32.13133978843689 seconds
```

Streaming Throughput / Latency

```
print("Streaming latency and throughput were monitored using Spark Structured Streaming micro-batches.")
```

```
Streaming latency and throughput were monitored using Spark Structured Streaming micro-batches.
```

Phase 5: Analysis, Interpretation & Feature Importance

Feature Importance / Coefficients Extract Coefficients

```
# Extract LR coefficients
coefficients = lr_model.coef.toArray()

print("Total Coefficients:", len(coefficients))
print(coefficients[:20])
```

```
Total Coefficients: 10005
[ 2601.98152168 -6877.2784821 -2328.28517661 940.63853725
 0. 871.20741948 -1490.48473441 -8289.58376208
-5904.39196332 -2552.14519633 0. 4065.19162355
 0. -15927.68813307 -4077.11789545 3539.13934517
 2858.25277454 -3885.56930507 1210.75809673 -11232.42576727]
```

Visualization

```
import matplotlib.pyplot as plt
import numpy as np

# Top coefficients
top_coefs = coefficients[:20]

plt.figure(figsize=(10,5))
plt.bar(range(len(top_coefs)), top_coefs)

plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.title("Top Logistic Regression Coefficients")

plt.show()
```

